



Universidade Federal de Sergipe — Departamento de Computação
COMP0498 — Algoritmos e Estruturas de Dados II · Turma T02 · 2026.2

Lista de Exercícios — Da análise de algoritmos à busca em largura

Prof. Dr. André Yoshiaki Kashiwabara · andreyoshiaki@dcomp.ufs.br

Convenções. Os grafos têm vértices $0, 1, \dots, V - 1$; V é o número de vértices e A o número de arcos. Um grafo não dirigido é um digrafo simétrico: cada aresta $\{v, w\}$ corresponde aos dois arcos $v \rightarrow w$ e $w \rightarrow v$. Nos rastreios, os vizinhos de um vértice são examinados em **ordem crescente**, qualquer que seja a representação (como se as listas de adjacência estivessem ordenadas). O tipo `Graph` é o das aulas: `struct graph` com campos `V`, `A` e `adj` — matriz `int **adj` em uma representação, vetor de listas `link *adj` na outra.

Parte I — Análise e corretude

Exercício 1. Custo de laços e recorrências.

Análise o custo de tempo dos três trechos abaixo em função de n , dando uma expressão assintótica justa (isto é, um Θ , não apenas um O).

```

1  /* A */                      /* B */
2  for (int i = 1; i <= n; i++)  for (int i = 1; i < n; i *= 2)
3      for (int j = i; j <= n; j++)  for (int j = 0; j < n; j++)
4          c++;                      c++;
5
6  /* C */
7  int f( int n) {
8      if (n <= 1) return 1;
9      int s = 0;
10     for (int i = 0; i < n; i++) s += i;    /* Theta(n) */
11     return s + f( n/2) + f( n/2);
12 }
```

- Para **A** e **B**, escreva o somatório que conta as execuções de `c++` e resolva-o.
- Escreva a recorrência $T(n)$ do trecho **C** e resolva-a pelo teorema mestre, identificando a , b , $f(n)$ e o caso aplicado.
- Resolva também $T(n) = T(n/2) + \Theta(1)$ e $T(n) = 2T(n/2) + \Theta(n^2)$, dizendo em cada uma qual caso do teorema se aplica e por quê.
- Dê uma recorrência que apareça em algum algoritmo visto na disciplina e para a qual o teorema mestre *não* se aplique; explique qual hipótese falha.

Exercício 2. Invariante do particionamento de Lomuto.

O trecho abaixo particiona $v[p..r]$ em torno do pivô $x = v[r]$.

```

1  int particiona( int v[], int p, int r) {
2      int x = v[r];
3      int i = p - 1;
4      for (int j = p; j < r; j++)
5          if (v[j] <= x) {
6              i++;
7              troca( &v[i], &v[j]);
8          }
9      troca( &v[i+1], &v[r]);
```

```

10  return i + 1;
11  }

```

- Enuncie o invariante do laço **for** descrevendo as **quatro regiões** do vetor em termos de p , i , j , r e do pivô x .
- Prove *inicialização*, *manutenção* (tratando os dois casos do **if**) e *término* do invariante.
- Use o invariante no momento do término para provar que, após a troca final, o valor devolvido q satisfaz $v[p..q-1] \leq v[q] \leq v[q+1..r]$.
- Rastreie a execução sobre $v = \{5, 2, 9, 1, 7, 3\}$ com $p = 0$ e $r = 5$, exibindo i , j e o vetor a cada iteração.
- Um colega trocou $v[j] \leq x$ por $v[j] < x$. O algoritmo ainda está correto? O que muda no invariante? Há diferença de desempenho quando o vetor tem muitos elementos iguais?

Exercício 3. Busca exaustiva e geração em ordem lexicográfica.

Uma mochila suporta capacidade C e há n itens, o item i com peso p_i e valor val_i . Queremos o subconjunto de valor máximo cujo peso total não ultrapassa C .

- Quantos subconjuntos existem? Escreva o custo da busca exaustiva em função de n , contando também o trabalho gasto para avaliar cada subconjunto.
- Implemente em C a enumeração de todos os subconjuntos de $\{0, 1, \dots, n-1\}$ em **ordem lexicográfica** — cada subconjunto visto como a sequência *crescente* dos seus elementos, de modo que $\{0, 1\}$ vem antes de $\{0, 2\}$ e $\{0, 3\}$ antes de $\{1\}$ — e use-a para resolver a mochila, devolvendo o melhor valor encontrado.
- Para $n = 4$, $C = 7$, pesos $\{5, 4, 3, 2\}$ e valores $\{10, 7, 6, 4\}$, liste a ordem em que os subconjuntos são gerados pela sua solução e indique qual deles vence.
- Escreva a função `proxima_permutacao(int v[], int n)`, que transforma v na permutação seguinte em ordem lexicográfica e devolve 0 quando v já é a última. Aplique-a três vezes seguidas a partir de $\{1, 3, 4, 2\}$, mostrando as três permutações obtidas.
- A força bruta é lenta, mas serve de *oráculo*. Descreva como você a usaria para testar uma solução mais rápida da mochila: que entradas geraria, de que tamanho, e o que compararia.

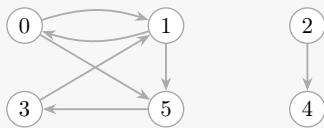
Exercício 4. Seleção e estatísticas de ordem.

Considere o vetor $v = \{9, 1, 8, 4, 6, 2, 7, 3\}$.

- Encontrar o **menor** elemento custa $n-1$ comparações. Mostre que esse número é ótimo, argumentando sobre quantos elementos precisam ser “derrotados” ao menos uma vez.
- O método ingênuo acha o menor e depois o menor dos restantes, com $(n-1) + (n-2) = 2n-3$ comparações. Escreva um algoritmo que ache o menor e o segundo menor com $n + O(\log n)$ comparações e dê quantas ele usa no pior caso em função de n . A partir de que n ele é estritamente melhor que o ingênuo?
- Implemente `int selecao(int v[], int p, int r, int k)`, que devolve o k -ésimo menor elemento de $v[p..r]$ ($1 \leq k \leq r-p+1$) usando o particionamento do Exercício 2 (*quickselect*), sem ordenar o vetor inteiro.
- Rastreie a sua `selecao` sobre o vetor acima com $k = 4$ (a mediana inferior), exibindo, a cada chamada, o intervalo considerado, o pivô e a posição devolvida pelo particionamento.
- Dê o custo do `quickselect` no melhor caso, no caso médio e no pior caso. Exiba uma permutação de $\{1, \dots, n\}$ e um valor de k que, juntos, forcem o pior caso quando o pivô é sempre o último elemento, e compare com a alternativa “ordenar e indexar”.

Parte II — Grafos e busca em profundidade

O **digrafo** G_1 , usado nos Exercícios 5 a 7, tem 6 vértices e os arcos 0-1, 1-0, 0-5, 1-5, 2-4, 3-1, 5-3:



Exercício 5. As duas representações e o que cada uma cobra.

Considere o digrafo G_1 acima.

- Escreva a matriz de adjacências 6×6 de G_1 e desenhe o vetor de listas de adjacência que resulta de inserir os arcos exatamente na ordem em que foram listados. Lembre que a inserção de um arco é feita no **início** da lista de adjacência, e não no fim.
- Dê o grau de entrada e o grau de saída de cada vértice. Qual soma vale A : a dos graus de entrada, a dos de saída, ou ambas? Justifique.
- Monte uma tabela com o custo de cada operação nas duas representações, em função de V , A e do grau $d(v)$: (i) decidir se o arco $v-w$ existe; (ii) percorrer todos os vizinhos de v ; (iii) percorrer todos os arcos do grafo; (iv) espaço ocupado.
- Um grafo tem $V = 10^5$ vértices e $A = 3 \times 10^5$ arcos. Qual representação você escolhe e por quê? E se fosse $A \approx V^2/2$?
- Escreva `int GRAPHoutdeg( Graph G, vertex v)` nas duas representações e comente a diferença de custo entre elas.

Exercício 6. Busca em profundidade: rastreo, floresta e a versão iterativa.

Execute a busca em profundidade sobre G_1 , examinando os vizinhos em ordem crescente.

- Rode `dfsR(G, 0)` e preencha a tabela de `pre[v]`, `post[v]` e `pai[v]`. Em seguida complete a varredura (`GRAPHdfs()`), que reinicia a busca nos vértices ainda não visitados, e desenhe a floresta DFS resultante indicando as raízes.
- Escreva a pré-ordem e a pós-ordem dos vértices. Usando o critério `pre[v] < pre[w]` e `post[v] > post[w]`, decida se 3 é descendente de 0 e se 4 é descendente de 0.
- Escreva a versão **iterativa** da busca, com uma pilha explícita na qual os vizinhos de cada vértice são *empilhados em ordem crescente*, e rastreie-a a partir de 0 mostrando o conteúdo da pilha a cada passo. A ordem de visita coincide com a da versão recursiva? Explique por quê, e diga o que seria preciso mudar para que coincidisse (ou deixasse de coincidir).
- Dê o custo da busca nas duas representações, separando o trabalho por vértice do trabalho por arco.
- Escreva `void caminho( Graph G, vertex s, vertex t)`, que execute *uma única* chamada `dfsR(G, s)` (sem a varredura) e, usando o vetor `pai[]` resultante, imprime um caminho de s até t , ou avisa que não existe. Aplique-a a $s = 0, t = 3$ e a $s = 0, t = 4$.

Exercício 7. Classificação de arcos, ciclos e componentes.

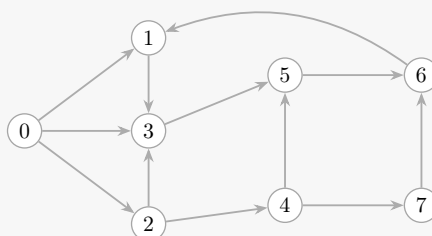
Acrescente a G_1 o arco 2-0, obtendo G'_1 .

- Verifique que `pre[]` e `post[]` não mudam com a inclusão de 2-0 e explique por quê.
- Classifique **todos** os arcos de G'_1 em arcos de árvore, de retorno, de avanço e cruzados, justificando cada um pelos valores de `pre[]` e `post[]`.

- (c) Enuncie e justifique o critério “um digrafo tem ciclo se e somente se a busca em profundidade encontra um arco de retorno”. Escreva `int temCiclo( Graph G)` baseada nele e diga seu custo.
- (d) G'_1 tem ciclo? Se tiver, exiba um. Em seguida, considere a versão **não dirigida** de G'_1 (os arcos 0-1 e 1-0 dão uma única aresta $\{0, 1\}$): por que, nesse caso, não existem arcos de avanço nem cruzados, e por que a detecção de ciclo precisa ignorar o arco pelo qual a busca chegou ao vértice?
- (e) Calcule o vetor `cc[]` da versão não dirigida de G_1 (isto é, sem o arco 2-0), dizendo quantos componentes conexos existem e quais são. O que muda quando o arco 2-0 é acrescentado?
- (f) Nos digrafos, o número de árvores da floresta DFS *não* responde quantos componentes fortemente conexos existem. Explique o problema com um exemplo tirado de G'_1 , descreva o algoritmo das duas buscas (Kosaraju–Sharir) e liste os componentes fortemente conexos de G'_1 .

Parte III — Busca em largura

O **digrafo** G_2 , usado nos Exercícios 8 a 10, tem 8 vértices e os arcos 0-1, 0-2, 0-3, 1-3, 2-3, 2-4, 3-5, 4-5, 5-6, 4-7, 7-6, 6-1:



Exercício 8. Busca em largura: rastreo e implementação.

A busca em largura visita primeiro todos os vértices a distância 1 da origem, depois todos os a distância 2, e assim por diante, usando uma **fila**.

- (a) Execute a busca em largura em G_2 a partir de 0, examinando os vizinhos em ordem crescente. Mostre, a cada iteração, o vértice retirado da fila e o conteúdo da fila depois de enfileirar os vizinhos novos.
- (b) Preencha a tabela de `d[v]` (número de arcos do caminho encontrado de 0 até v) e de `pai[v]`, e desenhe a árvore da busca. Organize o desenho em *camadas*: todos os vértices com o mesmo `d` na mesma linha.
- (c) Implemente `void bfs( Graph G, vertex s)` em C, na representação por **listas de adjacência**, com uma fila em vetor circular. Os vetores `d[]` e `pai[]` devem ficar preenchidos ao final.
- (d) Em que momento o vértice é marcado na sua implementação: ao ser *enfileirado* ou ao ser *desenfileirado*? Mostre, em G_2 , o que acontece de errado se a marcação for feita só na retirada.
- (e) Dê o custo da busca em largura nas duas representações e compare com o custo da busca em profundidade. Qual é o gasto de espaço da fila no pior caso?

Exercício 9. O que a busca em largura garante — e o que ela não garante.

- (a) Enuncie a propriedade central: ao final da busca em largura a partir de s , o valor `d[v]` é a *menor* quantidade de arcos de um caminho de s a v . Esboce a prova por indução sobre `d`, usando o invariante “em todo instante, a fila contém vértices de no máximo dois valores consecutivos de `d`, em ordem não decrescente”.

- (b) Rode a busca em *profundidade* em G_2 a partir de 0 e compare, na mesma tabela, o comprimento do caminho de 0 a cada vértice obtido pelas duas buscas. Aponte um vértice em que a busca em profundidade devolve um caminho estritamente mais longo do que o mínimo e explique por quê.
- (c) Se cada arco tivesse um *custo* (um número positivo qualquer), a busca em largura ainda daria o caminho mais barato? Dê um contraexemplo com três vértices e diga o que a busca em largura continua garantindo.
- (d) Na busca em largura sobre um grafo **não dirigido**, prove que nenhuma aresta liga um vértice a um ancestral ou descendente que não seja seu pai ou filho na árvore da busca (o análogo dos arcos de retorno e de avanço da busca em profundidade): toda aresta liga vértices de camadas iguais ou consecutivas, isto é, $|d[v] - d[w]| \leq 1$ para toda aresta $\{v, w\}$ entre vértices alcançados.
- (e) Use o item anterior para escrever `int bipartido( Graph G)`, que decide em tempo $\Theta(V + A)$ se um grafo não dirigido pode ter seus vértices pintados com duas cores sem que nenhum arco ligue dois vértices da mesma cor. Qual estrutura encontrada pela busca prova que o grafo *não* é bipartido?

Exercício 10. Escolher a busca certa.

Para cada problema abaixo, diga qual busca você usaria — em largura, em profundidade, ou tanto faz —, justifique em duas ou três frases e dê o custo da sua solução. Quando o enunciado pedir, escreva também o código.

- (a) Descobrir se existe caminho de s a t em um digrafo com milhões de vértices, sabendo que t , quando alcançável, costuma estar a poucos arcos de s .
- (b) Decidir se um digrafo tem ciclo.
- (c) Encontrar o menor número de movimentos para levar um jogo de um estado inicial a um estado final, sendo cada estado um vértice e cada movimento um arco. Explique por que o grafo não precisa existir na memória antes da busca.
- (d) Calcular, para um grafo não dirigido, o vetor `cc[]` de componentes conexos. Escreva a versão que usa a busca em largura como sub-rotina e compare-a com a versão recursiva vista em aula, do ponto de vista do consumo de *pilha* quando o grafo tem 10^6 vértices em um único caminho.
- (e) Determinar a *excentricidade* de um vértice s em um grafo não dirigido conexo — a maior distância de s a qualquer outro vértice. Escreva a função e diga o custo de calcular a excentricidade de *todos* os vértices.

Para estudar

Análise e recorrências: Cormen et al. (2009), cap. 3 e 4; Levitin (2012), §2.2–2.4.

Corretude e invariantes: Cormen et al. (2009), §2.1 e §7.1; Manber (1989), cap. 2; Gries (1981).

Força bruta, busca exaustiva e geração combinatória: Levitin (2012), cap. 3 e §4.3; Knuth (1998), §5.3.3.

Seleção: Hoare (1961), *Algorithm 65: FIND*; Cormen et al. (2009), cap. 9; Blum et al. (1973).

Grafos e representações: Feofiloff (2020), caps. *Grafos e Estruturas de dados para grafos*; Sedgewick (2002), cap. 17.

Busca em profundidade, arcos e componentes: Feofiloff (2020), cap. *Busca em profundidade*; Cormen et al. (2009), §22.3; Tarjan (1972); Sedgewick (2002), cap. 19.

Busca em largura: Feofiloff (2020), cap. *Busca em largura*; Cormen et al. (2009), §22.2; Sedgewick e Wayne (2011), §4.1; Moore (1959), *The shortest path through a maze*.